Security-Aware Multi-Client TCP Server

Overview

This project is a Python-based multi-client TCP server that allows multiple clients to connect to a central server and submit math requests over a structured JSON-based protocol. The server handles concurrent client sessions using threads, places incoming math requests into a shared FIFO queue, processes each request in a consistent order, and returns structured results to each client.

For GitHub and resume presentation, this project is best framed as a network programming and security-aware server project. It demonstrates TCP socket programming, client-server communication, protocol design, request logging, input handling, session tracking, and graceful connection termination.

Key Features

Feature	Description
Multi-client support	Multiple clients can connect to the server at the same time using separate client sessions.
Threaded server design	Each connected client is handled by the server while requests are coordinated through shared processing logic.
FIFO request handling	Math requests are placed into a shared queue so requests are processed in a consistent global order.
JSON line protocol	Client and server messages use one JSON object per line to keep TCP message boundaries clear.
Session tracking	The server records client joins, requests, responses, disconnects, and session duration for debugging and monitoring.
Security-aware controls	The improved GitHub version can include authentication, input validation, rate limiting, and audit-style logging.

Architecture Summary

The application follows a central server and multiple client model. A client starts a session by connecting to the server and sending a join message with a client name. The server responds with an acknowledgement message, then accepts math requests from the client. Each math request includes a message type, request identifier, and either a basic operation with operands or a full math expression. The server processes the request and returns a result or an error response. When the client is finished, it sends a close message and the server terminates the session cleanly.

Protocol Design

The project uses a simple JSON-based protocol over TCP. Each message is sent as text with one JSON object per line. This makes message parsing more reliable because the server can separate individual messages using line boundaries instead of guessing where one TCP message ends and the next begins.

Example message flow:

client connects -> join -> server ack -> math request -> result/error -> close -> bye

Programming Environment and Execution

The application was developed in Python using PyCharm for coding, testing, and debugging. It can be run from a terminal using the provided Makefile. The server listens on localhost and clients connect using the same IP address and port. Multiple client instances can be started in separate terminals to simulate concurrent users.

Example execution commands:

make server

make client

make client2

make client3

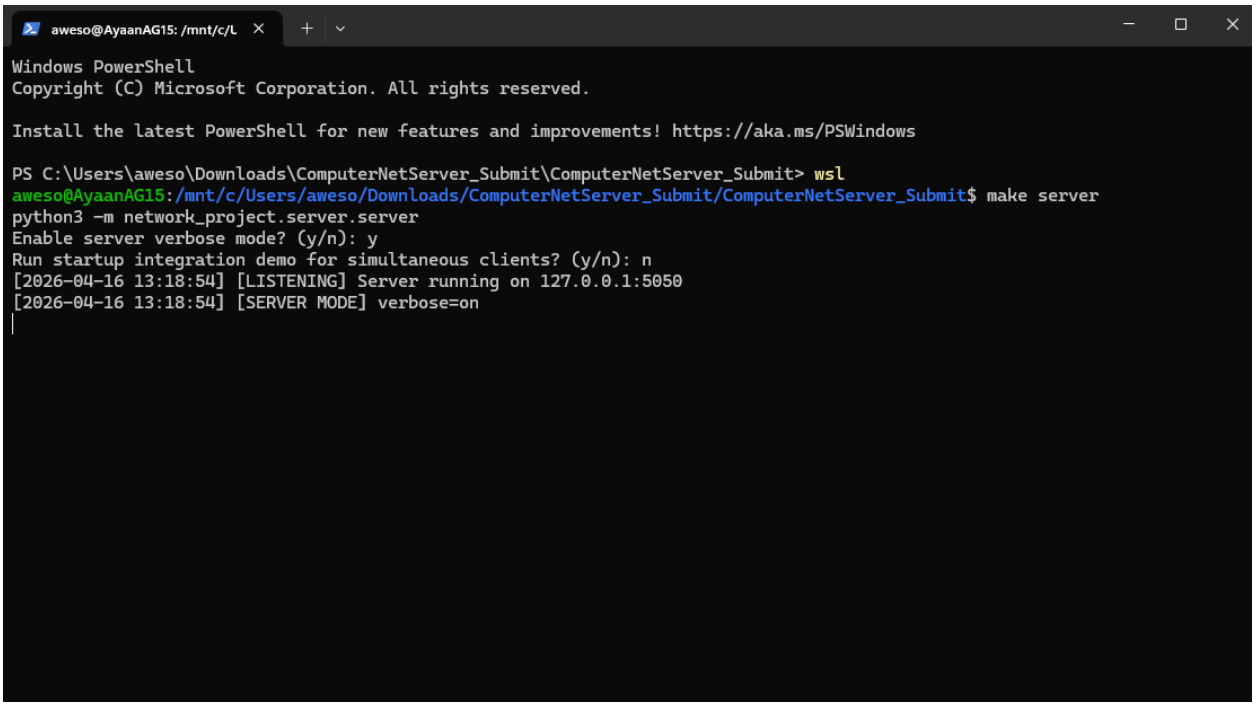
Skills Demonstrated

Python programming, TCP socket programming, Client-server architecture, Threading and concurrency, JSON protocol design, FIFO request handling, Input parsing, Request logging, Session management, Network troubleshooting, Technical documentation.

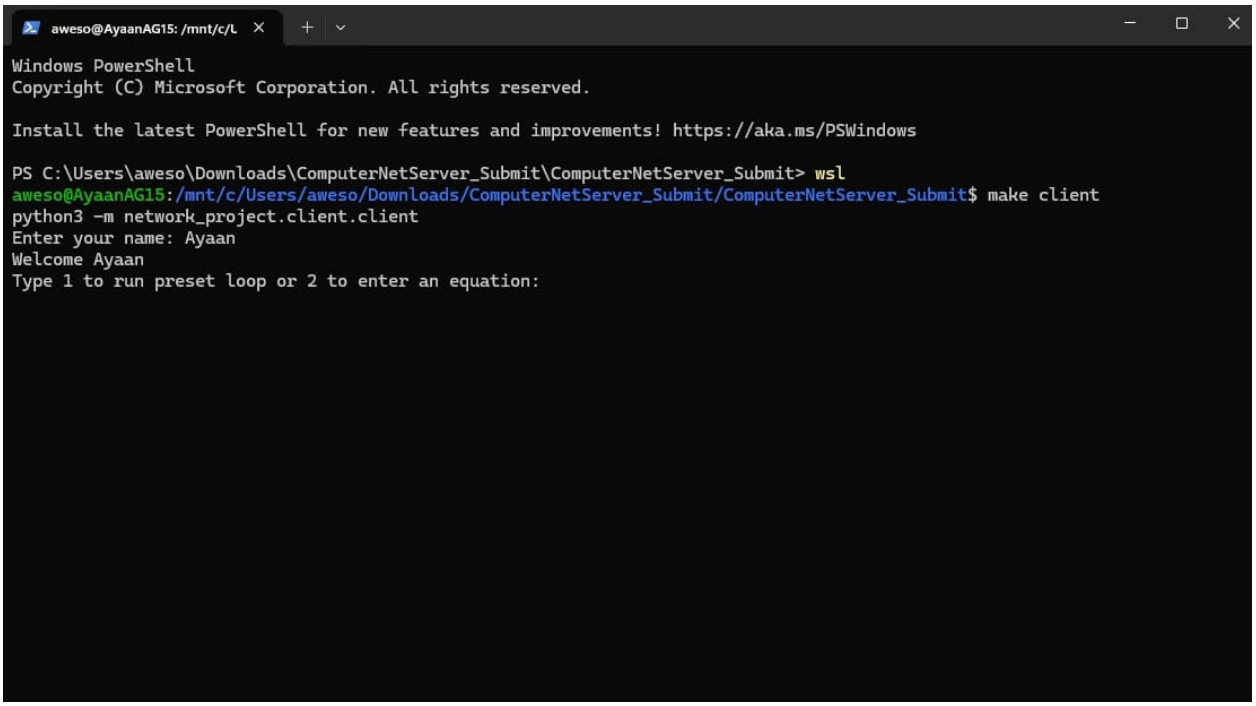
Testing and Verification

The application was tested using multiple client instances connected to the same server. The screenshots below show server startup, client startup, simultaneous client loops, complex expression handling, and preprogrammed math request loops.

Server-side Initialization



Client-side Initialization



Server - Simultaneous Client Loop

```
khatab@NoelStollen:/mnt/c/Users/Khata/OneDrive/Desktop/PC/ComputerNetServer_Submit$ make server
python3 -m network_project.server.server
Enable server verbose mode? (y/n): y
Run startup integration demo for simultaneous clients? (y/n): y
[2026-04-16 18:55:46] [LISTENING] Server running on 127.0.0.1:5050
[2026-04-16 18:55:46] [SERVER MODE] verbose=on
[2026-04-16 18:55:46] [INTEGRATION DEMO] enabled: two demo clients will join, send overlapping math requests, and close automatically.
[2026-04-16 18:55:46] [INTEGRATION DEMO] starting client threads: demo-alice and demo-bob
[2026-04-16 18:55:46] [NEW CONNECTION] ('127.0.0.1', 57070)
[2026-04-16 18:55:46] [NEW CONNECTION] ('127.0.0.1', 57072)
[2026-04-16 18:55:46] [JOIN] demo-bob joined from 127.0.0.1:57070 at 2026-04-16 18:55:46.884088
[2026-04-16 18:55:46] [JOIN] demo-alice joined from 127.0.0.1:57072 at 2026-04-16 18:55:46.887893
[2026-04-16 18:55:46] [INTEGRATION DEMO] demo-bob: joined successfully
[2026-04-16 18:55:46] [INTEGRATION DEMO] demo-alice: joined successfully
[2026-04-16 18:55:46] [REQUEST #1] demo-alice sent expr '2 + 2'
[2026-04-16 18:55:46] [RESPONSE #1] to demo-alice expr '2 + 2' -> 4
[2026-04-16 18:55:46] [INTEGRATION DEMO] demo-alice: req#1 expr '2 + 2' -> {'type': 'result', 'request_id': 1, 'status': 'ok', 'result': 4, 'analysis': {'n
ormalized_expression': '2+2', 'operands': [{'index': 1, 'value': 2.0}, {'index': 2, 'value': 2.0}], 'operators': [{'index': 1, 'symbol': '+', 'name': 'add'
}], 'counts': {'operand_count': 2, 'operator_count': 1}}}
[2026-04-16 18:55:46] [VERBOSE #1] normalized=2+2
[2026-04-16 18:55:46] [VERBOSE #1] counts operands=2 operators=1
[2026-04-16 18:55:46] [VERBOSE #1] operands [1:2.0, 2:2.0]
[2026-04-16 18:55:46] [VERBOSE #1] operators [1:add(+)]
[2026-04-16 18:55:47] [REQUEST #2] demo-bob sent expr '9 % 4'
[2026-04-16 18:55:47] [RESPONSE #2] to demo-bob expr '9 % 4' -> 1
[2026-04-16 18:55:47] [INTEGRATION DEMO] demo-bob: req#1 expr '9 % 4' -> {'type': 'result', 'request_id': 1, 'status': 'ok', 'result': 1, 'analysis': {'nor
malized_expression': '9%4', 'operands': [{'index': 1, 'value': 9.0}, {'index': 2, 'value': 4.0}], 'operators': [{'index': 1, 'symbol': '%', 'name': 'mod'}]
, 'counts': {'operand_count': 2, 'operator_count': 1}}}
[2026-04-16 18:55:47] [VERBOSE #2] normalized=9%4
[2026-04-16 18:55:47] [VERBOSE #2] counts operands=2 operators=1
[2026-04-16 18:55:47] [VERBOSE #2] operands [1:9.0, 2:4.0]
[2026-04-16 18:55:47] [VERBOSE #2] operators [1:mod(%)]
[2026-04-16 18:55:47] [REQUEST #3] demo-bob sent expr '10 / 2'
[2026-04-16 18:55:47] [RESPONSE #3] to demo-bob expr '10 / 2' -> 5.0
[2026-04-16 18:55:47] [INTEGRATION DEMO] demo-bob: req#2 expr '10 / 2' -> {'type': 'result', 'request_id': 2, 'status': 'ok', 'result': 5.0, 'analysis': {'
normalized_expression': '10/2', 'operands': [{'index': 1, 'value': 10.0}, {'index': 2, 'value': 2.0}], 'operators': [{'index': 1, 'symbol': '/', 'name': 'd
iv'}], 'counts': {'operand_count': 2, 'operator_count': 1}}}
[2026-04-16 18:55:47] [REQUEST #4] demo-alice sent expr '(8-3) * 2'
[2026-04-16 18:55:47] [VERBOSE #3] normalized=10/2
[2026-04-16 18:55:47] [VERBOSE #3] counts operands=2 operators=1
[2026-04-16 18:55:47] [VERBOSE #3] operands [1:10.0, 2:2.0]
[2026-04-16 18:55:47] [VERBOSE #3] operators [1:div(/)]
[2026-04-16 18:55:47] [RESPONSE #4] to demo-alice expr '(8-3) * 2' -> 10
[2026-04-16 18:55:47] [INTEGRATION DEMO] demo-alice: req#2 expr '(8-3) * 2' -> {'type': 'result', 'request_id': 2, 'status': 'ok', 'result': 10, 'analysis'
: {'normalized_expression': '(8-3)*2', 'operands': [{'index': 1, 'value': 8.0}, {'index': 2, 'value': 3.0}, {'index': 3, 'value': 2.0}], 'operators': [{'in
dex': 1, 'symbol': '-', 'name': 'sub'}, {'index': 2, 'symbol': '*', 'name': 'mul'}], 'counts': {'operand_count': 3, 'operator_count': 2}}}
[2026-04-16 18:55:47] [DISCONNECTED] demo-bob from 127.0.0.1:57070 | Duration: 0.41s | Requests: 2
[2026-04-16 18:55:47] [INTEGRATION DEMO] demo-bob: close -> {'type': 'bye', 'message': 'Goodbye demo-bob'}
[2026-04-16 18:55:47] [VERBOSE #4] normalized=(8-3)*2
[2026-04-16 18:55:47] [VERBOSE #4] counts operands=3 operators=2
[2026-04-16 18:55:47] [VERBOSE #4] operands [1:8.0, 2:3.0, 3:2.0]
[2026-04-16 18:55:47] [VERBOSE #4] operators [1:sub(-), 2:mul(*)]
[2026-04-16 18:55:47] [DISCONNECTED] demo-alice from 127.0.0.1:57072 | Duration: 0.43s | Requests: 2
[2026-04-16 18:55:47] [INTEGRATION DEMO] demo-alice: close -> {'type': 'bye', 'message': 'Goodbye demo-alice'}
[2026-04-16 18:55:47] [INTEGRATION DEMO] completed
```

Complex Equations - Server and Three Clients

```
khatab@NoelStollen:/mnt/c/Users/Khata/OneDrive/Desktop/PC/ComputerNetServer_Submit$ make server
python3 -m network_project.server.server
Enable server verbose mode? (y/n): y
Run startup integration demo for simultaneous clients? (y/n): n
[2026-04-16 19:04:56] [LISTENING] Server running on 127.0.0.1:5050
[2026-04-16 19:04:56] [SERVER MODE] verbose=on
[2026-04-16 19:05:05] [NEW CONNECTION] ('127.0.0.1', 59446)
[2026-04-16 19:05:14] [NEW CONNECTION] ('127.0.0.1', 40446)
[2026-04-16 19:05:17] [JOIN] joe joined from 127.0.0.1:40446 at 2026-04-16 19:05:17.912887
[2026-04-16 19:05:20] [JOIN] rae joined from 127.0.0.1:59446 at 2026-04-16 19:05:20.152552
[2026-04-16 19:06:02] [NEW CONNECTION] ('127.0.0.1', 42472)
[2026-04-16 19:06:09] [JOIN] west joined from 127.0.0.1:42472 at 2026-04-16 19:06:09.671079
[2026-04-16 19:07:37] [REQUEST #1] rae sent expr '(17 % 5) + 6'
[2026-04-16 19:07:37] [RESPONSE #1] to rae expr '(17 % 5) + 6' -> 8
[2026-04-16 19:07:37] [VERBOSE #1] normalized=(17%5)+6
[2026-04-16 19:07:37] [VERBOSE #1] counts operands=3 operators=2
[2026-04-16 19:07:37] [VERBOSE #1] operands [1:17.0, 2:5.0, 3:6.0]
[2026-04-16 19:07:37] [VERBOSE #1] operators [1:mod(%), 2:add(+)]
[2026-04-16 19:07:53] [REQUEST #2] joe sent expr '(6 + 3) * 4 - 5'
[2026-04-16 19:07:53] [RESPONSE #2] to joe expr '(6 + 3) * 4 - 5' -> 31
[2026-04-16 19:07:54] [VERBOSE #2] normalized=(6+3)*4-5
[2026-04-16 19:07:54] [VERBOSE #2] counts operands=4 operators=3
[2026-04-16 19:07:54] [VERBOSE #2] operands [1:6.0, 2:3.0, 3:4.0, 4:5.0]
[2026-04-16 19:07:54] [VERBOSE #2] operators [1:add(+), 2:mul(*), 3:sub(-)]
[2026-04-16 19:08:08] [REQUEST #3] west sent expr '(9 - 15) / 3 + 8'
[2026-04-16 19:08:08] [RESPONSE #3] to west expr '(9 - 15) / 3 + 8' -> 6.0
[2026-04-16 19:08:08] [VERBOSE #3] normalized=(9-15)/3+8
[2026-04-16 19:08:08] [VERBOSE #3] counts operands=4 operators=3
[2026-04-16 19:08:08] [VERBOSE #3] operands [1:9.0, 2:15.0, 3:3.0, 4:8.0]
[2026-04-16 19:08:08] [VERBOSE #3] operators [1:sub(-), 2:div(/), 3:add(+)]
```

```
PS C:\Users\Khata\OneDrive\Desktop\PC\ComputerNetServer_Submit> wsl
khatab@NoelStollen:/mnt/c/Users/Khata/OneDrive/Desktop/PC/ComputerNetServer_Submit$ make client
python3 -m network_project.client.client
Enter your name: rae
Welcome rae
Type 1 to run preset loop or 2 to enter an equation: 2
Enter equations like: add 5 3, 8 % 3, or (2-3)/2 + 2
Type 'close' to disconnect at any time.
Equation 1: (17 % 5) + 6
Result: 8
Send another equation? (y/n):
```

```
khatab@NoelStollen:/mnt/c/Users/Khata/OneDrive/Desktop/PC/ComputerNetServer_Submit$ make client2
python3 -m network_project.client.client2
Enter your name: joe
Welcome joe
Type 1 to run preset loop or 2 to enter an equation: 2
Enter equations like: add 5 3, 8 % 3, or (2-3)/2 + 2
Type 'close' to disconnect at any time.
Equation 1: (6 + 3) * 4 - 5
Result: 31
Send another equation? (y/n):
```

```
khatab@NoelStollen:/mnt/c/Users/Khata/OneDrive/Desktop/PC/ComputerNetServer_Submit$ make client3
python3 -m network_project.client.client3
Enter your name: west
Welcome west
Type 1 to run preset loop or 2 to enter an equation: 2
Enter equations like: add 5 3, 8 % 3, or (2-3)/2 + 2
Type 'close' to disconnect at any time.
Equation 1: (9 - 15) / 3 + 8
Result: 6.0
Send another equation? (y/n):
```

Preprogrammed Loop Test

```
khatab@NoelStollen:/mnt/c/Users/Khata/OneDrive/Desktop/PC/ComputerNetServer_Submit$ make server
python3 -m network_project.server.server
Enable server verbose mode? (y/n): y
Run startup integration demo for simultaneous clients? (y/n): n
[2026-04-16 19:13:18] [LISTENING] Server running on 127.0.0.1:5050
[2026-04-16 19:13:18] [SERVER MODE] verbose=on
[2026-04-16 19:13:26] [NEW CONNECTION] ('127.0.0.1', 47010)
[2026-04-16 19:13:31] [JOIN] kade joined from 127.0.0.1:47010 at 2026-04-16 19:13:31.608421
[2026-04-16 19:13:37] [REQUEST #1] kade sent add [27, 35]
[2026-04-16 19:13:37] [RESPONSE #1] to kade add [27, 35] -> 62
[2026-04-16 19:13:37] [VERBOSE #1] normalized=27+35
[2026-04-16 19:13:37] [VERBOSE #1] counts operands=2 operators=1
[2026-04-16 19:13:37] [VERBOSE #1] operands [1:27.0, 2:35.0]
[2026-04-16 19:13:37] [VERBOSE #1] operators [1:add(+)]
[2026-04-16 19:13:38] [REQUEST #2] kade sent mul [12, 6]
[2026-04-16 19:13:38] [RESPONSE #2] to kade mul [12, 6] -> 72
[2026-04-16 19:13:38] [VERBOSE #2] normalized=12*6
[2026-04-16 19:13:38] [VERBOSE #2] counts operands=2 operators=1
[2026-04-16 19:13:38] [VERBOSE #2] operands [1:12.0, 2:6.0]
[2026-04-16 19:13:38] [VERBOSE #2] operators [1:mul(*)]
[2026-04-16 19:13:39] [REQUEST #3] kade sent mod [41, 35]
[2026-04-16 19:13:39] [RESPONSE #3] to kade mod [41, 35] -> 6
[2026-04-16 19:13:39] [VERBOSE #3] normalized=41%35
[2026-04-16 19:13:39] [VERBOSE #3] counts operands=2 operators=1
[2026-04-16 19:13:39] [VERBOSE #3] operands [1:41.0, 2:35.0]
[2026-04-16 19:13:39] [VERBOSE #3] operators [1:mod(%)]
[2026-04-16 19:13:40] [REQUEST #4] kade sent div [6, 45]
[2026-04-16 19:13:40] [RESPONSE #4] to kade div [6, 45] -> 0.13333333333333333
[2026-04-16 19:13:40] [VERBOSE #4] normalized=6/45
[2026-04-16 19:13:40] [VERBOSE #4] counts operands=2 operators=1
[2026-04-16 19:13:40] [VERBOSE #4] operands [1:6.0, 2:45.0]
[2026-04-16 19:13:40] [VERBOSE #4] operators [1:div(/)]
[2026-04-16 19:13:42] [REQUEST #5] kade sent add [6, 3]
[2026-04-16 19:13:42] [RESPONSE #5] to kade add [6, 3] -> 9
[2026-04-16 19:13:42] [VERBOSE #5] normalized=6+3
[2026-04-16 19:13:42] [VERBOSE #5] counts operands=2 operators=1
[2026-04-16 19:13:42] [VERBOSE #5] operands [1:6.0, 2:3.0]
[2026-04-16 19:13:42] [VERBOSE #5] operators [1:add(+)]
[2026-04-16 19:13:42] [REQUEST #6] kade sent div [31, 44]
```

```
khatab@NoelStollen:/mnt/c/Users/Khata/OneDrive/Desktop/PC/ComputerNetServer_Submit$ make client
python3 -m network_project.client.client
Enter your name: kade
Welcome kade
Type 1 to run preset loop or 2 to enter an equation: 1
Running preset loop with 20 random equations...
Equation 1: 27 + 35
Result: 62
Equation 2: 12 * 6
Result: 72
Equation 3: 41 % 35
Result: 6
Equation 4: 6 / 45
Result: 0.13333333333333333
Equation 5: 6 + 3
Result: 9
Equation 6: 31 / 44
Result: 0.70454545454545546
Equation 7: 3 % 15
Result: 3
Equation 8: 43 * 46
Result: 1978
Equation 9: 48 + 5
Result: 53
Equation 10: 48 + 22
Result: 70
Equation 11: 16 / 46
Result: 0.34782608695652173
Equation 12: 34 % 23
Result: 11
Equation 13: 11 * 31
Result: 341
Equation 14: 48 * 7
Result: 336
```

Challenges and Lessons Learned

The main challenge was making sure multiple clients could connect, send requests, receive responses, and disconnect cleanly without breaking the server. Working with sockets also required careful handling of message boundaries, session state, and connection cleanup. This project helped strengthen practical understanding of how network applications communicate, how servers handle multiple clients, and why structured protocols and logging are important for reliability and troubleshooting.